Machine Learning Basics

Instructors: Mohammadreza A. Dehaqani, Babak n. Arabi, Mostafa Tavassolipour

Saman Davachi Tousi, Paria Pasehvarz

Fall 2025

Homework 2

Question 1: Parzen Windows

Let $p(x) \sim U(0, a)$ be uniform from 0 to a, and let a Parzen window be defined as

$$\phi(x) = e^{-x} \quad \text{for } x > 0 \quad \text{and} \quad 0 \text{ for } x \leq 0.$$

1. Show that the mean of such a Parzen-window estimate is given by:

$$\bar{p}_n(x) = \begin{cases} 0, & x < 0, \\ \frac{1}{a}\left(1 - e^{-x/h_n}\right), & 0 \leq x \leq a, \\ \frac{1}{a}\left(e^{a/h_n} - 1\right) e^{-x/h_n}, & a \leq x. \end{cases}$$

2. Plot $\bar{p}_n(x)$ versus x for $a = 1$ and $h_n = 1,\ 0.25,$ and 0.0625.
3. How small does h_n have to be to have less than 1% bias over 99% of the range $0 < x < a$?
4. Find h_n for this condition if $a = 1$, and plot $\bar{p}_n(x)$ in the range $0 \leq x \leq 0.05$.

Question 2: Shift of the Eigenvalue Spectrum

In this problem, analyzing the regularization mechanism of ridge regression will serve as an excuse to do some linear algebra.

1. Let $A, B \in \mathbb{R}^{n \times n}$ be symmetric matrices. Assume that $\xi \in \mathbb{R}^n$ is an eigenvector for *both* matrices, with eigenvalues λ_A, λ_B respectively. Please show that ξ is an eigenvector of $A + B$. What is the corresponding eigenvalue?
2. Now consider the ridge regression solution

$$\hat{\beta}^{\text{ridge}} = (X^\top X + \lambda I)^{-1} X^\top y.$$

   Please use the result in 1.) to explain why computing $\hat{\beta}^{\text{ridge}}$ is more stable than $\hat{\beta}$, i.e., why the solution can be reliably computed (for a suitable λ) even if $X^\top X$ is numerically singular.
3. Compare linear regression and ridge regression in terms of the bias–variance trade-off.

Question 3: Optimizing a Kernel

Kernel-based regression techniques are similar to nearest-neighbor regressors: rather than fit a parametric model, they predict values for new data points by interpolating values from existing points in the training set. In this problem, we will consider a kernel-based regressor of the form

$$f(x^*) = \frac{\sum_n K(x_n, x^*) y_n}{\sum_n K(x_n, x^*)},$$

where (x_n, y_n) are the training data points, and $K(x, x')$ is a kernel function that defines the similarity between two inputs x and x'. Assume that each x_i is represented as a row vector, i.e., a $1 \times D$ vector where D is the number of features for each data point. A popular choice of kernel is a function that decays as the distance between the two points increases, such as

$$K(x, x') = \exp\big(-\|x - x'\|_2^2\big) = \exp\big(-(x - x')(x - x')^\top\big)\,.$$

However, the squared Euclidean distance $\|x - x'\|_2^2$ may not always be the right choice. In this problem, we will consider optimizing over squared Mahalanobis distances

$$K(x, x') = \exp\big(-(x - x')W(x - x')^\top\big)\,,$$

where W is a symmetric $D \times D$ matrix. Intuitively, introducing the weight matrix W allows for different dimensions to matter differently when defining similarity.

1. Let $\{(x_n, y_n)\}_{n=1}^N$ be our training data set. Suppose we are interested in minimizing the residual sum of squares. Write down this loss over the training data $L(W)$ as a function of W.

   Hint: For each point (x_i, y_i) in the training dataset, consider for which subset of training data you should sum kernel function K over for each $f(x_i)$.

2. In the following, let us assume that $D = 2$. That means that W has three parameters: W_{11}, W_{22}, and $W_{12} = W_{21}$. Expand the formula for the loss function to be a function of these three parameters.

Question 4: KNN and the Curse of Dimensionality

When the number of features p is large, there tends to be a deterioration in the performance of KNN and other *local* approaches that perform prediction using only observations that are near the test observation for which a prediction must be made. This phenomenon is known as the *curse of dimensionality*, and it ties into the fact that parametric approaches often perform poorly when p is large. We will now investigate this curse.

In each of the following scenarios, we assume that each predictor is uniformly (evenly) distributed on $[0, 1]$, and that each observation is associated with a response value.

(a) Suppose that we have a set of observations, each with measurements on $p = 1$ feature, X. Suppose that we wish to predict a test observation's response using only observations that are within 10% of the range of X closest to that test observation. For instance, in order to predict the response for a test observation with $X = 0.6$, we will use observations in the range $[0.55, 0.65]$. On average, **what fraction of the available observations** will we use to make the prediction?

(b) Now suppose that we have a set of observations, each with measurements on $p = 2$ features, X_1 and X_2. We wish to predict a test observation's response using only observations that are within 10% of the range of X_1 and within 10% of the range of X_2 closest to that test observation. On average, **what fraction of the available observations** will we use to make the prediction?

(c) Now suppose that we have a set of observations on $p = 100$ features. We wish to predict a test observation's response using observations within the 10% of each feature's range that is closest to that test observation. **What fraction of the available observations** will we use to make the prediction?

(d) Using your answers to parts (a)–(c), argue that a drawback of KNN when p is large is that there are very few training observations "near" any given test observation.

Question 5: Choosing Metrics

Suppose you wanted to classify a mysterious drink as coffee, energy drink, or soda based on the amount of caffeine and amount of sugar per 1-cup serving. Coffee typically contains a large amount of caffeine, lemonade typically contains a large amount of sugar, and energy drinks typically contain a large amount of both. Which distance metric would work best, and why? Answer 1, 2, 3, or 4:

1. Euclidean distance, because you're comparing standard numeric quantities

2. Manhattan distance, because it makes sense to array the training data points in a grid
3. Hamming distance, because the features "contains caffeine" and "contains sugar" are boolean values
4. Cosine distance, because you care more about the ratio of caffeine to sugar than the actual amount

Suppose you add a fourth possible classification, water, which contains no caffeine or sugar. Now which distance metric would work best, and why? Answer 1, 2, 3, or 4:

1. Euclidean distance, because you're still comparing standard numeric quantities, and it's now more difficult to compare ratios
2. Manhattan distance, because there are now four points, so it makes even more sense to array them in a grid
3. Hamming distance, because the features "contains caffeine" and "contains sugar" are still boolean values
4. Cosine distance, because you still care more about the ratio of caffeine to sugar than the actual amount

While visiting Manhattan, you discover that there are 3 different types of taxis, each of which has a distinctive logo. Each type of taxi comes in many makes and models of cars. Which distance metric would work best for classifying additional taxis based on their logos, makes, and models? Answer 1, 2, 3, or 4:

1. Euclidean distance, because there's a lot of variability in make and model
2. Manhattan distance, because the streets meet at right angles (and because you're in Manhattan and classifying taxicabs)
3. Hamming distance, because the features are non-numeric
4. Cosine distance, because it will separate taxis by how different their makes and models are

Why might an ID tree work better for classifying the taxis from previous question ? Answer 1, 2, 3, or 4:

1. k-nearest neighbors requires data to be graphed on a coordinate system, but the training data isn't quantitative
2. Some taxis may have the same make and model, and k-nearest neighbors can't handle identical training points
3. An ID tree can ignore irrelevant features, such as make and model
4. Taxis aren't guaranteed to stay within their neighborhood in Manhattan

Question 6: Kernel Ridge Regression

In contrast to ordinary least squares which has a cost function

$$J(\theta) = \frac{1}{2}\sum_{i=1}^{m}(\theta^\top x^{(i)} - y^{(i)})^2,$$

we can also add a term that penalizes large weights in θ. In *ridge regression*, our least squares cost is regularized by adding a term $\lambda\|\theta\|^2$, where $\lambda > 0$ is a fixed (known) constant. The ridge regression cost function is then

$$J(\theta) = \frac{1}{2}\sum_{i=1}^{m}(\theta^\top x^{(i)} - y^{(i)})^2 + \frac{\lambda}{2}\|\theta\|^2.$$

(a) Use the vector notation to find a closed-form expression for the value of θ which minimizes the ridge regression cost function.

(b) Suppose that we want to use kernels to implicitly represent our feature vectors in a high-dimensional (possibly infinite dimensional) space. Using a feature mapping ϕ, the ridge regression cost function becomes

$$J(\theta) = \frac{1}{2}\sum_{i=1}^{m}(\theta^\top \phi(x^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2}\|\theta\|^2.$$

Making a prediction on a new input x_{new} would now be done by computing $\theta^\top \phi(x_{\text{new}})$. Show how we can use the "kernel trick" to obtain a closed form for the prediction on the new input without ever explicitly computing $\phi(x_{\text{new}})$. You may assume that the parameter vector θ can be expressed as a linear combination of the input feature vectors; i.e.,

$$\theta = \sum_{i=1}^{m} \alpha_i \phi(x^{(i)})$$

for some set of parameters α_i.

Hint: You may find the following identity useful:

$$(\lambda I + BA)^{-1}B = B(\lambda I + AB)^{-1}.$$

If you want, you can try to prove this as well, though this is not required for the problem.

Question 7: Deriving Linear Regression

We noted that the solution for the least squares linear regressions "looked" like a ratio of covariance and variance terms. In this problem, we will derive that. Let us assume the following generative process for our data:

$$x \sim N(0, \Sigma_x)$$

$$\epsilon \sim N(0, \sigma^2)$$

$$y \mid x, \epsilon \;=\; w^\top x + \epsilon$$

Assume scalar x, c, w, and y, and that x is independent of ϵ.

1. Provide a formula for $\Sigma_{yx} = E_{x,y}[yx]$ based on the above generative model.
2. Provide a formula to estimate $E_{x,y}[yx]$ given observed data $\{(x_n, y_n)\}_{n=1}^N$.
3. Moment terms like $E_{x,y}[yx]$, $E_{x,y}[xx^\top]$, etc. can easily be estimated from the data (like you did above). Write down an expression for the optimal w^* which minimizes expected squared residual loss in terms of moments (e.g. $\mu_x, \Sigma_x, \Sigma_{yx}, \sigma$).
4. Now, suppose the data x were generated from $N(\mu_x, \Sigma_x)$. Write an expression for w^* in terms of moments (as above).
5. Would the formula for w^* derived in (part 4) hold if the process generating the x's were no longer Gaussian, but still had the same means, variances, and covariances? That is, $E[x] = \mu_x$, $\text{Var}[x] = \Sigma_x$, $\text{Var}[\epsilon] = \sigma^2$, and $E[yx] = \Sigma_{yx}$, but the distribution of x is not Gaussian?

Question 8: Logistic Regression with Newton's Method

Given examples $\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n \in \mathbb{R}^d$ and associated labels $y_1, y_2, \ldots, y_n \in \{0, 1\}$, the cost function for *unregularized* logistic regression is

$$J(\mathbf{w}) \triangleq -\sum_{i=1}^{n} \left(y_i \ln s_i + (1 - y_i)\ln(1 - s_i)\right)$$

where $s_i \triangleq s(\mathbf{x}_i \cdot \mathbf{w})$, $\mathbf{w} \in \mathbb{R}^d$ is a weight vector, and

$$s(y) \triangleq \frac{1}{1 + e^{-y}}$$

is the logistic function.

Define the $n \times d$ design matrix X (whose ith row is $\mathbf{x}_i^\top$), the label n-vector $\mathbf{y} \triangleq [y_1 \ \dots \ y_n]^\top$, and $\mathbf{s} \triangleq [s_1 \ \dots \ s_n]^\top$. For an n-vector $\mathbf{a}$, let

$$\ln \mathbf{a} \triangleq [\ln a_1 \ \dots \ \ln a_n]^\top.$$

The cost function can be rewritten in vector form as

$$J(\mathbf{w}) = -\mathbf{y} \cdot \ln \mathbf{s} - (1 - \mathbf{y}) \cdot \ln(1 - \mathbf{s}).$$

Further, recall that for a real symmetric matrix $A \in \mathbb{R}^{d \times d}$, there exist U and Λ such that $A = U \Lambda U^\top$ is the eigendecomposition of A. Here Λ is a diagonal matrix with entries $\{\lambda_1, \dots, \lambda_d\}$. An alternative notation is $\Lambda = \operatorname{diag}(\lambda_i)$, where diag() takes as input the list of diagnal entries, and constructs the corresponding diagonal matrix. This notation is widely used in libraries like numpy and is useful for simplifying some of the expressions when written in matrix–vector form. For example, we can write $\mathbf{s} = \operatorname{diag}(s_i)\mathbf{1}$.

Hint: See page two for notational conventions used here.

Hint: Recall matrix calculus identities. The elements in ***bold*** *indicate vectors.*

$$\nabla_x \alpha y = (\nabla_x \alpha)\, y^\top + \alpha \nabla_x y \qquad \nabla_x (y \cdot z) = (\nabla_x y) z + (\nabla_x z) y;$$

$$\nabla_x f(y) = (\nabla_x y)(\nabla_y f(y)); \qquad \nabla_x g(y) = (\nabla_x y)(\nabla_y g(y));$$

$$\text{and} \qquad \nabla_x C y(x) = (\nabla_x y(x)) C^\top, \quad \text{where } C \text{ is a constant matrix.}$$

1. Derive the gradient $\nabla_{\mathbf{w}} J(\mathbf{w})$ of cost $J(\mathbf{w})$ as a matrix–vector expression. Also derive *all intermediate derivatives* in matrix–vector form. Do *not* specify them (including the intermediates) in terms of their individual components (e.g., w_i). You are *only* allowed to use individual components if and only if they are inside a diag function.

2. Derive the Hessian $\nabla^2_{\mathbf{w}} J(\mathbf{w})$ for the cost function $J(\mathbf{w})$ as a matrix–vector expression.

3. Write the matrix–vector update law for one iteration of Newton's method, substituting the gradient and Hessian of $J(\mathbf{w})$.

4. You are given four examples

   $$\mathbf{x}_1 = [0.2 \ \ 3.1]^\top, \quad \mathbf{x}_2 = [1.0 \ \ 3.0]^\top, \quad \mathbf{x}_3 = [-0.2 \ \ 1.2]^\top, \quad \mathbf{x}_4 = [1.0 \ \ 1.1]^\top$$

   with labels $y_1 = 1, y_2 = 1, y_3 = 0, y_4 = 0$. These points cannot be separated by a line passing through the origin. Hence, as described in lecture, append a 1 to each $\mathbf{x}_i$ and use a weight vector $\mathbf{w} \in \mathbb{R}^3$ whose last component is the bias term (called α in lecture).

   Begin with initial weight

   $$\mathbf{w}^{(0)} = [-1 \ \ 1 \ \ 0]^\top.$$

   For the following, state only the final answer with four digits after the decimal point. You may use a calculator or write a program to solve for these, but **do NOT** submit any code for this part.

Question 9: A Bayesian Interpretation of Lasso

Suppose you are aware that the labels y_i corresponding to sample points $x_i \in \mathbb{R}^d$ follow the density law

$$f(y_i \mid x_i, \mathbf{w}) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(y_i - \mathbf{w} \cdot x_i)^2}{2\sigma^2}\right)$$

where $\sigma > 0$ is a known constant and $\mathbf{w} \in \mathbb{R}^d$ is a random parameter. Suppose further that experts have told you that:

- Each component of $\mathbf{w}$ is independent of the others, and

- Each component of $\mathbf{w}$ has the Laplace distribution with location 0 and scale being a known constant b. That is, each component w_j obeys the density law

$$f(w_j) = \frac{1}{2b} \exp\left(-\frac{|w_j|}{b}\right)$$

Assume the outputs y_i are independent from each other.

Your goal is to find the choice of parameter $\mathbf{w}$ that is *most likely* given the input-output examples (x_i, y_i). This method of estimating parameters is called *maximum a posteriori* (MAP); Latin for *"maximum [odds] from what follows."*

1. Derive the *posterior* probability density law $f(\mathbf{w} \mid (x_i, y_i))$ for $\mathbf{w}$ *up to a proportionality constant* by applying Bayes' Theorem and substituting for the densities $f(y_i \mid x_i, \mathbf{w})$ and $f(\mathbf{w})$. Don't try to derive an exact expression for $f(\mathbf{w} \mid (x_i, y_i))$, as the denominator is very involved and irrelevant to maximum likelihood estimation.

2. Define the log-likelihood for MAP as

$$\ell(\mathbf{w}) \triangleq \ln f(\mathbf{w} \mid (x_i, y_i))$$

Show that maximizing the MAP log-likelihood over all choices of $\mathbf{w}$ is the same as minimizing

$$\sum_{i=1}^{n} (y_i - \mathbf{w} \cdot x_i)^2 + \lambda \|\mathbf{w}\|_1$$

where $\|\mathbf{w}\|_1 = \sum_{j=1}^{d} |w_j|$ and λ is a constant. Also give a formula for λ as a function of the distribution parameters.

Question 10: Parzen Window Classification with Gaussian Kernel

Consider Parzen-window estimates and classifiers for points in the table below. Let your window function be a spherical Gaussian, i.e.,

$$\phi\left(\frac{\mathbf{x} - \mathbf{x}_i}{h}\right) \propto \exp\left[-\frac{(\mathbf{x} - \mathbf{x}_i)^\top (\mathbf{x} - \mathbf{x}_i)}{2h^2}\right].$$

(a) Write a program to classify an arbitrary test point $\mathbf{x}$ based on the Parzen window estimates. Train your classifier using the three-dimensional data from your three categories in the table above. Set $h = 1$ and classify the following three points:

$$(0.50,\ 1.0,\ 0.0)^\top, \quad (0.31,\ 1.51,\ -0.50)^\top, \quad \text{and} \quad (-0.3,\ 0.44,\ -0.1)^\top.$$

(b) Repeat with $h = 0.1$.

Sample	ω_1 (x_1, x_2, x_3)	ω_2 (x_1, x_2, x_3)	ω_3 (x_1, x_2, x_3)
1	0.28, 1.31, -6.2	0.011, 1.03, -0.21	1.36, 2.17, 0.14
2	0.07, 0.58, -0.78	1.27, 1.28, 0.08	1.41, 1.45, -0.38
3	1.54, 2.01, -1.63	0.13, 3.12, 0.16	1.22, 0.99, 0.69
4	-0.44, 1.18, -4.32	-0.21, 1.23, -0.11	2.46, 2.19, 1.31
5	-0.81, 0.21, 5.73	-2.18, 1.39, -0.19	0.68, 0.79, 0.87
6	1.52, 3.16, 2.77	0.34, 1.96, -0.16	2.51, 3.22, 1.35
7	2.20, 2.42, -0.19	-1.38, 0.94, 0.45	2.60, 2.44, 0.92
8	0.91, 1.94, 6.21	-0.12, 0.82, 0.17	0.64, 0.13, 0.97
9	0.65, 1.93, 4.38	-1.44, 2.31, 0.14	0.85, 0.58, 0.99
10	-0.26, 0.82, -0.96	0.26, 1.94, 0.08	0.66, 0.51, 0.88

Table 1: Three-dimensional data samples for categories ω_1, ω_2, and ω_3

Question 11: Maximum likelihood estimation

1. The file normal.data.txt has a random sample from a normal distribution.

   (a) Find the maximum likelihood estimates of $\hat{\mu}$ and $\hat{\sigma}^2$ numerically. Compare the answer to your closed-form solution.

   (b) Show that the minus log-likelihood is indeed minimized at $(\hat{\mu}, \hat{\sigma}^2)$ for this data set.

   (c) Calculate the estimated asymptotic covariance matrix of the MLEs.

   (d) Give a "better" estimated asymptotic covariance matrix based on your closed-form solution.

   (e) Calculate a large-sample 95% confidence interval for σ^2.

   (f) Test $H_0 : \mu = 103$ with a

      i. Z-test.

      ii. Likelihood ratio chi-squared test. Compare the closed-form version.

      iii. Wald chi-squared test.

      Give the test statistic and the p-value for each test.

   (g) The coefficient of variation (used in sample surveys and business statistics) is the standard deviation divided by the mean.

      i. Show that multiplication by a positive constant does not affect the coefficient of variation. This is a paper-and-pencil calculation.

      ii. Give a numerical point estimate of the coefficient of variation for the normal data of this question. Actually, it's the maximum likelihood estimate, because the invariance principle of maximum likelihood estimation says that the MLE of a function is that function of the MLE.

      iii. Using the delta method, give a 95% confidence interval for the coefficient of variation. Start with a paper-and-pencil calculation of $g(\theta) = \left(\frac{\partial g}{\partial \theta_1}, \ldots, \frac{\partial g}{\partial \theta_k}\right)$.

Question 12: Non-Linear Regression using basis functions

In this problem, we consider a way of extending the linear regression technique to non-linear problems. A simple (but often effective) approach is linear regression with basis expansion. Instead of actually fitting a non-linear function to the data, the data is preprocessed by a non-linear mapping, and linear regression is then applied to the transformed problem. This method is equivalent to fitting a regression function of the form

$$\hat{f}(x) = \sum_{j=0}^{d} \beta_j h_j(x), \tag{12.1}$$

where the basis functions h_j are fixed. The linear coefficients β_j are the target parameters of the learning problem. The parameter β_0 accounts for the bias, and the corresponding basis function is the constant function $h_0(x) = 1$.

To apply linear regression in the one-dimensional case, we transform each observation $x^{(i)}$ into a vector $h(x^{(i)}) := (h_0(x^{(i)}), \ldots, h_d(x^{(i)}))$. The input data vector $x = (x^{(1)}, \ldots, x^{(n)})^\top$ is then substituted by a matrix H, containing the vectors $h(x^{(i)})$ as its rows:

$$\begin{bmatrix} x^{(1)} \\ x^{(2)} \\ \vdots \\ x^{(n)} \end{bmatrix} \quad \text{becomes} \quad \begin{bmatrix} h_0(x^{(1)}) & h_1(x^{(1)}) & \cdots & h_d(x^{(1)}) \\ h_0(x^{(2)}) & h_1(x^{(2)}) & \cdots & h_d(x^{(2)}) \\ \vdots & \vdots & & \vdots \\ h_0(x^{(n)}) & h_1(x^{(n)}) & \cdots & h_d(x^{(n)}) \end{bmatrix}.$$

Both standard linear regression and ridge regression are now applicable.

Here is what we would ask you to do:

1. Generate values from a sinc function
$$f(x) = \text{sinc}(3x),$$
by uniformly sampling the input domain and adding zero-mean Gaussian noise to the function values, to obtain:
$$y = f(x) + \epsilon, \tag{12.2}$$
where $x \sim \text{Uniform}(0, 1)$ and $\epsilon \sim \mathcal{N}(0, 0.01)$. The input vector x consists of $n = 25$ samples and the corresponding output vector is y.

2. Program a regression estimator. The basis functions we consider are Gaussians of the form
$$h_j(x) = \exp\left(-\frac{(x - \mu_j)^2}{2\sigma_j^2}\right), \tag{12.3}$$
for $j = 1, \ldots, d$. Program an estimation routine
$$\beta = \text{regress.gauss}(x, y, \text{means}, \text{var}, \lambda)$$
The arguments are:
   - x — a single observation vector
   - y — the corresponding outputs
   - `means` — vector of basis function means μ_j
   - `var` — the (scalar) variance parameter of the basis functions
   - `lambda` — the ridge regression regularization parameter
   - `beta` — the estimate $\hat{\beta}$ or $\hat{\beta}^{\text{ridge}}$, respectively.

3. Apply your estimator to the data you generated. Use the following values for the parameters:
$$d = 21 \quad \text{(a total of 22 functions including the bias)},$$
$$\mu_j \text{ equally spaced over the interval } [0, 1],$$
$$\sigma_j = 0.04,$$
$$\lambda \in \{0.001, 0.05, 5\}.$$
For each λ, plot a graph that shows:
   - the function estimated from training vector x,
   - the function estimated by averaging over the $\beta^{(i)}$ obtained from repeating the above experiment 100 times (i.e. generating pairs $(x^{(i)}, y^{(i)})$, estimating coefficients $\beta^{(i)}$, and averaging over all runs to obtain β^{avg}).

   The estimated function is given by
$$\hat{f}(x) = \hat{\beta}_0^{\text{avg}} + \sum_{j=1}^{d} h_j(x)\, \hat{\beta}_j^{\text{avg}}. \tag{12.4}$$
   To plot the function, compute $\hat{f}(x)$ for $x \in [0, 1]$ equally spaced (e.g. using `linspace`). Comment about the plots and the quality of solutions.

4. The estimation error can be computed by looking at the squared error of the estimator to the true function. Please plot the average error as a function of λ. To this end, repeat the estimation process for the 100 random vectors $x^{(i)}, i = 1, \ldots, 100$, for the regularizations
$$\lambda = \exp(\{-5 : 0.3 : 2\}).$$
   Instead of computing the L_2 distance between the functions by integration, we approximate it by
$$\text{err} = \frac{1}{m} \sum_{i=1}^{m} \left(f(z_i) - \hat{f}(z_i)\right)^2, \tag{12.5}$$
   where $z_1, \ldots, z_m$ are equally spaced points ($m = 100$). Plot the average errors to show how it varies over λ. Comment on what happens on the two ends of the plot.